

Paidly — Product Architecture Strategy

Systems + Document Engine + Experience layer · with technical appendix

PAIDLY ARCHITECTURE V2 (CLEAN REFERENCE)

One-page upgrade summary. The Part A outline below also names **Relationship & Offering** separately—the client/catalog **input graph** that feeds the Document Engine.

Paidly is structured around **four core engines**:

1. **Identity System** — Auth, users, organizations, roles, RLS-backed tenancy.
2. **Document Engine** — Invoices, quotes, payslips unified as `document(type=...)` ; shared compose, send, and PDF paths.
3. **Financial Engine** — Payments, cash flow, reports, subscriptions (reads and rails downstream of documents).
4. **Growth Engine** — Affiliates, email, notifications, CRM-style behaviour (how Paidly scales).

Frontend layout system

Same structure everywhere:

- **Header** — title + primary actions
- **Content** — tables, grids, main forms
- **Side panel** — summary, filters, secondary controls (`PageTemplate` in code)

Data flow (client):

UI → Hooks → Services → Entity facades (EntityManager) → Supabase

Serverless APIs (`/api/*`) handle:

- Payments (**Payfast**)
- Emails (**Resend** / SMTP)
- Public document access (shares, tokens)
- **Cron** jobs (dunning, reminders, ops)

Goal: Evolve Paidly from an **invoicing tool** into a full **business operating system**.

EXECUTIVE FRAMING

Old lens	New lens
Separate Invoices / Quotes / Payslips	One Document Engine , multiple document kinds
Features as silos	Four core systems + cross-cutting Experience System
Routes drive thinking	Capabilities drive architecture; routes are entry points

Part A — Core systems

1. Identity & Access

Supabase Auth, profiles, orgs, memberships, roles, RLS. **Strategy:** stabilise—harden session, org bootstrap, `RequireAuth` so other systems trust it cheaply.

2. Document Engine (commerce core)

Job: Issue documents to third parties → deliver → track → settle. **Kinds today:** Invoice, Quote, Payslip (each maps to existing entities/tables).

Shared lifecycle: Author → Compose → Render (PDF) → Deliver (email/link) → Observe (opens, reminders) → Settle (pay / accept / archive).

Product point: Invest once in send, PDF, public token, status, and line-item patterns—parity across kinds is *engine work*, not three copies.

3. Relationship & Offering

Clients + catalog (`services`) + line intelligence—**input graph** into the Document Engine, not a disconnected “contacts app.”

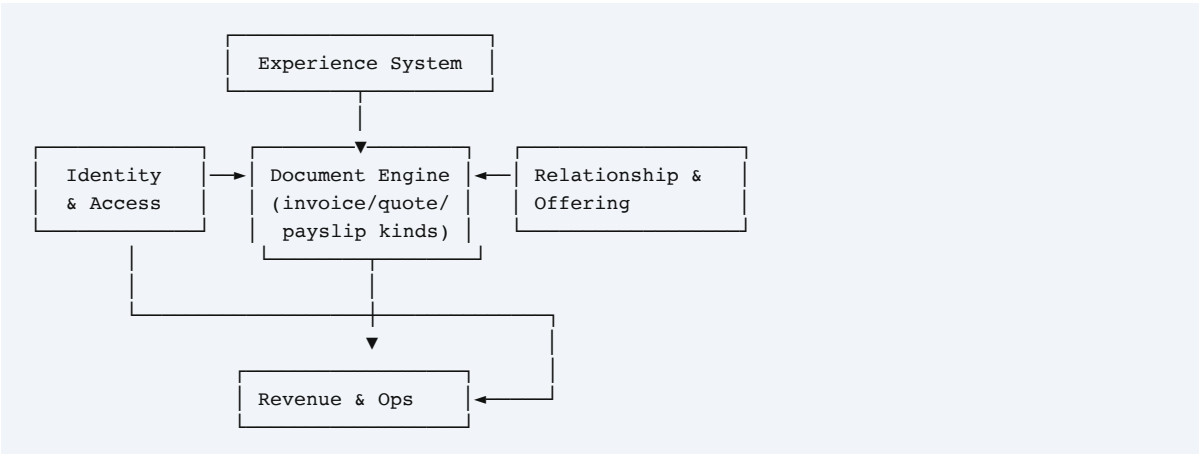
4. Revenue & Operating

Payfast, subscriptions, crons, affiliates, admin, reporting. **Strategy:** keep thin on rules that belong inside documents (e.g. overdue logic with invoices).

5. Experience System (cross-cutting)

Pillar	Meaning
Shell	Editor pattern: work area + sticky summary/actions (EditQuote / EditClient / EditCatalogItem)
Tokens	Semantic colours—border-border, bg-card, muted-foreground
Data discipline	Query keys, invalidation, cache-first hydration where safe
A11y	Label/id parity, meaningful disabled states

SYSTEM MAP



Part B — Technical appendix

Stack: Vite + React + React Router · Tailwind + Radix · TanStack Query · Supabase · Vercel `/api/*` · Payfast · Resend · Anvil · Turnstile.

Repo: `src/pages/` (routes), `src/services/` (domain), `src/api/customClient.js` (EntityManager → Supabase), `api/` (serverless), `supabase/migrations/`.

Flow: Auth → org scope → UI → Hooks → Services → EntityManager → Supabase + Query cache; `/api/*` for Payfast, email, public shares, crons.

FOUNDATION PRIORITIES & NEXT STEPS BACKLOG

Foundation §4–5 (see full `Paidly-Application-Blueprint.md`): **4** = stabilize Auth + Session; **5** = standardize UI layout. They are explicit in architecture **A.1 / A.2** and in execution below. Backlog **4–5** map directly to Foundation §4–5. Item **6** (Experience checklist) **closes** Foundation §5. Item **7** (role matrix) **supports** Foundation §4. Former items 6–9 are renumbered **8–11** (same substance: `PageTemplate` rollout, hook→service pattern, Client Timeline, quote→invoice).

Next steps (engineering backlog 1–11)

1. Grow `src/document-engine/` — status enums, send + PDF adapters, thin facades over Invoice / Quote / Payslip (supports High impact §1).

2. Line up quote / invoice / payslip on shared deliver + observe interfaces; compose converges on Create Document + type (High impact §1).
3. Make Financial Engine consumers explicit — cash flow / reports through document-shaped views (Client Timeline + money story).
4. **Stabilize Auth + Session** — execute Foundation §4: session read/write matrix, invites, org bootstrap, runbook/ADR.
5. **Standardize UI layout** — execute Foundation §5 + A.2: `PageTemplate`, editor shell, embedded admin.
6. Publish **Experience checklist** (1 page) — Page Template three-zone rule + layout grammar row picker (**closes** Foundation §5).
7. **Identity / role matrix doc** — admin vs management vs support (complements Foundation §4).
8. Roll out `PageTemplate` on Invoices, Quotes, Clients, Services, Affiliates — filters/summary into `sidePanel`.
9. Extend **hook → service → entity** pattern — `*ListService` for Quotes, Clients, etc.; keep `api/InvoiceService` -style modules for delivery.
10. **Client Timeline** — query + UI on Client detail (High impact §2); optional `ClientTimelineService`.
11. **Quote → invoice conversion** — draft invoice from accepted quote (High impact §3); RLS + idempotency.

Product-led priorities (document unification, client timeline, conversion flow) live in the blueprint section **High impact next**.